



Kaynak Yönetimi

Kubernetes ve Konteyner Dünyası



Bu Konuda

- Request ve limit kararları, miktar birimleri
- Yerleşim hesabı ve node allocatable
- CPU kısılması ve bellek OOM davranışı
- In-place resize ve Pod düzeyi kaynaklar
- Yerel geçici depolama
- QoS sınıfları ve namespace politikaları
- `kubectl top` ve Metrics Server

Request ve Limit





İki Karar: Yerleşim ve Tüketim

Karar anı	Ana sinyal	Sonuç
Pod nereye yerleşebilir?	<code>request</code> ve node <code>allocatable</code>	Scheduler uygun node arar
Çalışırken ne kadar tüketebilir?	<code>limit</code> ve kernel mekanizmaları	CPU kısılr; bellek baskısında süreç sonlandırılabilir

- `request`, scheduler'ın yerleşim hesabıdır; kaynak boşsa kullanım request değerini aşabilir, çalışma anındaki etkisi kaynak türüne göre değişir^[1]
- `limit`, node'da kapasite ayırmaz; çalışan konteyner için üst sınır davranışını tanımlar^[1]



Miktar Birimleri

```
resources:
  requests:
    cpu: 100m
    memory: 64Mi
  limits:
    cpu: 500m
    memory: 128Mi
```

- `1` CPU bir fiziksel ya da sanal çekirdeğe; `100m`, `0.1` CPU'ya karşılık gelir. CPU hassasiyeti `1m` değerinden daha küçük olamaz^[1]
- Bellek bayt cinsindendir. `Mi` / `Gi` ikinin kuvvetlerini, `M` / `G` onluk birimleri gösterir; küçük `m`, milli-bayttır ve çoğunlukla yazım hatasıdır^[1]
- Yalnız `limit` verilirse Kubernetes aynı değeri genel olarak `request` olarak kopyalar^[1]
- İstenen kapasite ayrı, izin verilen tepe ayrı değerlendirilir; her ikisi de ölçümle gerekçelendirilir



Yerleşim Hesabı: Request Toplamı

Yeni Pod'un etkili request'i + yerleşmiş Pod request'leri $\leq$ node allocatable

- `capacity`, node'un toplam CPU, bellek ve Pod kapasitesidir; `allocatable`, normal Pod'ların kullanabileceği bölümdür^[10]
- Sistem bileşenleri ve eviction eşiği için ayrılan bölüm nedeniyle `allocatable`, genellikle `capacity` değerinden küçüktür; scheduler bu kullanılabilir bütçeyi aşmaz^[11]
- Node boş görünse bile request toplamı sığmıyorsa Pod `Pending` kalır; `FailedScheduling` `Insufficient cpu` / `memory` nedenini gösterir. `kubectl top` bu hesabı değil, kullanımı gösterir^[1]

```
kubectl get node -o custom-columns='NAME:.metadata.name,CPU:.status.capacity.cpu,ALLOC:.status.allocatable.cpu'
kubectl describe pod pending-demo
```



CPU Limiti ve Kısılma

- Linux'ta `container runtime`, kubelet'in verdiği CPU değerlerini cgroup ayarlarına dönüştürür; kernel limiti uygular^[1]
- CPU `request`, çekişme olduğunda görelî ağırlık sağlar; boş kapasite varken konteyner request değerinin üstüne çıkabilir^[1]
- CPU `limit`, zaman dilimindeki CPU payı tükenince cgroup'u bekletir. Bu **CPU kısılmasıdır ("throttling")**; aşırı CPU kullanımı tek başına Pod'u sonlandırmaz^[1]

```
request: 100m    limit: 500m
  yerleşim      en çok yaklaşık yarım çekirdek CPU zamanı
```

Teşhis: uygulama çalışıyor ama gecikme artıyorsa yalnız CPU kullanımı değil, CPU limiti ve kısılma metrikleri de incelenir



Bellek Limiti ve OOMKilled

1. Süreç bellek ayırır; kullanım `request` değerini geçebilir
2. Kullanım limiti aşar ve kernel bellek baskısını algılayarsa OOM mekanizması bir süreci sonlandırabilir^[1]
3. Konteynerin ana süreci sonlanırsa durum geçmişinde `reason: OOMKilled` görülebilir; Kubernetes örneğinde buna `exitCode: 137` eşlik eder^[1]
4. Ana süreç yeniden başlatılabiliyorsa kubelet konteyneri yeniden başlatır; sonuç Pod'un `restartPolicy` değeriyle birlikte yorumlanır^[1]

```
kubectl get pod memory-demo -o jsonpath='{.status.\ncontainerStatuses[*].lastState.terminated}'
```

- `137` tek başına nedeni kanıtlamaz; OOM tanısı için `reason: OOMKilled`, limit, olaylar ve uygulama davranışı birlikte okunur

In-Place Resize ve Pod Düzeyi Kaynaklar





In-Place Resize

Kubernetes v1.36'da `InPlacePodVerticalScaling` kararlı ve varsayılan olarak açıktır; çalışan Pod'daki konteynerlerin CPU ve bellek `requests` / `limits` değerleri `/resize` alt kaynağı üzerinden güncellenebilir.^[2,5]

```
kubectl patch pod resize-demo --subresource=resize -p '{
  "spec": {"containers": [{"name": "pause", "resources": {
    "requests": {"cpu": "800m"}, "limits": {"cpu": "800m"}}}]}'
```

- İstenen değerler `spec.containers[*].resources`, çalışan konteynere uygulanmış değerler `status.containerStatuses[*].resources` alanındadır^[2]
- Workload şablonunu değiştirmek ise controller'ın yeni Pod oluşturduğu taşınabilir yöntemdir; doğrudan Pod resize, Deployment şablonunu güncellemez^[1]



Resize Status ve Politika

Alan	Anlam
<code>PodResizePending / Infeasible</code>	İstek bu node'da mümkün değil
<code>PodResizePending / Deferred</code>	Şimdilik uygulanamadı; kubelet yeniden dener
<code>PodResizeInProgress</code>	Kaynak ayrıldı, değişiklik henüz tamamen uygulanmadı
<code>resizePolicy.restartPolicy</code>	<code>NotRequired</code> varsayılandır; <code>RestartContainer</code> yeniden başlatmaya izin verir

- Pod `restartPolicy: Never` ise `RestartContainer` kullanılamaz^[2]
- Yalnız CPU ve bellek resize edilebilir; Windows Pod'ları, yeniden başlatılamayan init konteynerler ve ephemeral konteynerler desteklenmez^[2]
- Resize, Pod oluşturulurken belirlenen QoS sınıfını değiştiremez; yeni değerler mevcut sınıfın kurallarını korumalıdır^[2]
- Bellek limitini yeniden başlatmadan azaltmak yarış koşulu taşır; kullanım yeni limitin üstündeyse kubelet değişikliği bekletebilir^[2]



Pod Düzeyi Kaynaklar

```
spec:
  resources:
    requests: {cpu: 500m, memory: 256Mi}
    limits: {cpu: "1", memory: 512Mi}
  containers:
  - {name: app, image: registry.k8s.io/pause:3.10.1}
  - {name: helper, image: registry.k8s.io/pause:3.10.1}
```

- `PodLevelResources`, v1.36'da **beta ve varsayılan açıktır**; `spec.resources` Pod'un CPU, bellek ve hugepages toplamını tanımlar. Windows Pod'ları desteklenmez^[3,5]
- Pod ve konteyner düzeyi birlikte verilirse Pod düzeyi request/limit ve QoS hesabında önceliklidir; konteynerler Pod sınırı içindeki boş kapasiteyi paylaşabilir^[3]
- Pod düzeyi in-place resize için **ayrı** `InPlacePodLevelResourcesVerticalScaling` kapısı gerekir (v1.36 beta/açık); `PodLevelResources`, `InPlacePodVerticalScaling` ve `NodeDeclaredFeatures` da önkoşuldur^[4,5]

Yerel Geçici Depolama ve QoS





Yerel Geçici Depolama

```
resources:
  requests: {ephemeral-storage: 200Mi}
  limits: {ephemeral-storage: 1Gi}
volumes:
- name: scratch
  emptyDir: {sizeLimit: 500Mi}
- name: scratch-memory
  emptyDir: {sizeLimit: 50Mi, medium: Memory}
```

- Disk tabanlı `emptyDir`, yazılabilir konteyner katmanı ve Pod logları yerel geçici depolama tüketimine katılır; `tmpfs` `emptyDir` ise bellek kullanımı sayılır^[6]
- Scheduler `ephemeral-storage` request toplamını node kapasitesiyle karşılaştırır. Ölçüm desteklenen kubelet dosya sistemi düzenine bağlıdır^[6]
- Ölçülen kullanım konteyner veya Pod limitini aşarsa kubelet Pod'u eviction için işaretler; ölçüm yoksa bu limite dayalı eviction gerçekleşmez^[6]
- Depolama miktarlarında da `400m`, 0.4 bayttır; çoğu durumda amaçlanan değer `400Mi` olur^[6]



QoS Sınıfları

QoS	CPU ve bellek koşulu	Node baskısındaki konumu
Guaranteed	Her konteynerde request = limit > 0; Pod düzeyi kullanılıyorsa iki kaynak da Pod request = limit	En son tercih edilen sınıf
Burstable	En az bir CPU/bellek request veya limit var; Guaranteed koşulu yok	BestEffort sonrasında
BestEffort	Pod ve tüm konteynerlerde CPU/bellek request ve limit yok	İlk tercih edilen sınıf

- QoS, eviction kararlarını etkiler; tek başına hayatta kalma garantisi değildir. Kaynak baskısında request değerini aşan Pod'lar aday olur^[8]
- Bir konteyner bellek limitini aşıp OOM müdahalesine uğrarsa QoS sınıfı bunu engellemez^[1,8]
- QoS sınıfı Pod oluşturulurken belirlenir ve yaşamı boyunca değişmez; in-place resize sınıfı değiştirecekse reddedilir^[8]

```
kubectl get pod qos-demo -o jsonpath='{.status.qosClass}'
```

Namespace Politikaları ve Uygulama





LimitRange ve ResourceQuota

Politika	Kapsam	Yanıtladığı soru
LimitRange	Tek Pod, konteyner veya PVC	Bir nesnenin min/max/default/limit-request'i uygun mu?
ResourceQuota	Namespace toplamı	Tüm nesnelerin toplam isteği, limiti veya sayısı bütçeyi aşıyor mu?

- İki mekanizma kabul aşamasında yeni veya güncellenen nesneyi denetler; sonradan yapılan politika değişikliği mevcut nesneleri geriye dönük değiştirmez [\[12,13\]](#)



LimitRange

```
apiVersion: v1
kind: LimitRange
metadata: {name: compute-policy}
spec:
  limits:
  - type: Container
    default: {cpu: 500m, memory: 256Mi}
    defaultRequest: {cpu: 100m, memory: 64Mi}
    min: {cpu: 50m, memory: 32Mi}
    max: {cpu: "1", memory: 512Mi}
```

- `default` limit, `defaultRequest` request ekler; `min`, `max` ve `maxLimitRequestRatio` kabul sınırlarıdır^[12]
- İhlal eden Pod isteği `403 Forbidden` ve sınırı açıklayan mesajla reddedilir^[12]
- Aynı namespace'teki birden çok LimitRange varsa hangi varsayılanın uygulanacağı belirli değildir; tek, tutarlı varsayılan politikası tercih edilir^[12]
- Enjekte edilen varsayılanların kendi aralarındaki tutarlılığı otomatik garanti edilmez; request <= limit koşulu politika tasarımında doğrulanır^[12]



ResourceQuota

```
apiVersion: v1
kind: ResourceQuota
metadata: {name: team-budget}
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 2Gi
    limits.cpu: "4"
    limits.memory: 4Gi
    pods: "10"
```

- Quota, terminal olmayan Pod'ların toplam CPU/bellek request ve limitlerini; ayrıca `pods`, `services` gibi nesne sayılarını sınırlandırabilir^[13]
- Quota aşımı isteği `403 Forbidden` ile reddeder. Deployment nesnesi kabul edilse bile onun oluşturacağı Pod'lar quota nedeniyle reddedilebilir^[13]
- Quota, cluster kapasitesinden bağımsız mutlak muhasebedir; kapasite ayırmaz ve namespace quota toplamalarının cluster kapasitesini aşmasını engellemez^[13]

```
kubectl describe resourcequota team-budget -n team-a
```



kubectl top ve Metrics Server

```
kubectl top node  
kubectl top pod -n team-a  
kubectl top pod web -n team-a --containers
```

- `kubectl top`, Metrics API üzerinden node ve Pod CPU/bellek kullanımını gösterir; yeni Pod metrikleri veri hattı gecikmesi nedeniyle birkaç dakika görünmeyebilir^[14,16]
- Metrics Server, kubelet'lerden resource metrics toplar ve `metrics.k8s.io` API'sini HPA/VPA ile `kubectl top` için sunar; cluster'da ayrıca kurulmuş olmalıdır^[14]
- CPU, bir zaman penceresindeki ortalama çekirdek kullanımınıdır; bellek, toplama anındaki tahmini çalışma kümesi ("working set") değeridir^[14]
- Herhangi bir gösterge paneli ("dashboard"), alarm veya uygulama metriği sağlanmaz^[15]

Kaynaklar





Kaynaklar

1. Resource Management for Pods and Containers: kubernetes.io/docs/concepts/configuration/manage-resources-containers/
2. Resize CPU and Memory Resources Assigned to Containers: kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources/
3. Assign Pod-level CPU and Memory Resources: kubernetes.io/docs/tasks/configure-pod-container/assign-pod-level-resources/
4. Resize CPU and Memory Resources Assigned to Pods: kubernetes.io/docs/tasks/configure-pod-container/resize-pod-resources/
5. Feature Gates: kubernetes.io/docs/reference/command-line-tools-reference/feature-gates/
6. Local Ephemeral Storage: kubernetes.io/docs/concepts/storage/ephemeral-storage/
7. Device Plugins: kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/
8. Pod Quality of Service Classes: kubernetes.io/docs/concepts/workloads/pods/pod-qos/
9. Dynamic Resource Allocation: kubernetes.io/docs/concepts/scheduling-eviction/dynamic-resource-allocation/



Kaynaklar (devam)

- 10. Node Status: kubernetes.io/docs/reference/node/node-status/
- 11. Reserve Compute Resources for System Daemons: kubernetes.io/docs/tasks/administer-cluster/reserve-compute-resources/
- 12. Limit Ranges: kubernetes.io/docs/concepts/policy/limit-range/
- 13. Resource Quotas: kubernetes.io/docs/concepts/policy/resource-quotas/
- 14. Resource Metrics Pipeline: kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/
- 15. Tools for Monitoring Resources: kubernetes.io/docs/tasks/debug/debug-cluster/resource-usage-monitoring/
- 16. kubectl top: kubernetes.io/docs/reference/kubectl/generated/kubectl_top/
- 17. kubectl get: kubernetes.io/docs/reference/kubectl/generated/kubectl_get/
- 18. kubectl run: kubernetes.io/docs/reference/kubectl/generated/kubectl_run/